

1. Descripción General

La capa **Network** define el protocolo de comunicación y la infraestructura de socket TCP necesaria para permitir la interacción bidireccional y asíncrona en tiempo real entre el servidor central y los clientes del juego **Monopoly TEC**.

Esta capa abstrae la transmisión de datos serializando los mensajes en formato **JSON** con delimitador de salto de línea (\n), lo que garantiza la entrega ordenada y libre de fragmentación de cada paquete de red.

2. Componentes y Arquitectura de Clases

Network.ClienteTcp

Clase encargada de administrar el socket TcpClient y el flujo de datos (NetworkStream) en el lado del cliente.

- **Establecimiento de Conexión (Conectar):** Inicia la conexión síncrona hacia la dirección IP y puerto del servidor. Al conectarse exitosamente, arranca un hilo secundario (Background Thread) para escuchar el canal de entrada de forma continua sin bloquear el hilo de la interfaz gráfica.
- **Procesamiento y Acumulación de Tramas (EscucharServidor):**
 - Lee el flujo de bytes en un buffer de 4 KB y utiliza un StringBuilder para acumular los fragmentos de texto entrantes.
 - Resuelve el problema de fragmentación e hiper-empaquetamiento de TCP buscando activamente el carácter salto de línea (\n).
 - Extrae cada línea completa, la deserializa mediante Mensaje.Deserializar y dispara el evento MensajeRecibido.
- **Envío de Mensajes (Enviar):** Serializa la instancia de Mensaje, concatena el delimitador \n, la convierte a codificación **UTF-8** y la transmite por el NetworkStream.
- **Manejo de Desconexión (Cerrar):** Se encarga de cerrar de manera limpia el socket y el stream de datos, disparando el evento Desconectado.

3. Protocolo de Comunicación (Protocolo.cs)

El protocolo de red define la estructura estandarizada de los mensajes e intercambios de datos entre nodos.

Clase Mensaje

Representa el envoltorio genérico (wrapper) para todas las comunicaciones del sistema.

- **Tipo (TipoMensaje):** Enumeración que especifica la acción u orden del mensaje.
- **Datos (object?):** Contenedor genérico para la carga útil (*payload*).
- **LeerDatos<T>():** Método genérico que permite deserializar dinámicamente la propiedad Datos al tipo de *payload* específico que corresponde al TipoMensaje.

Enumeración TipoMensaje

Divide los tipos de comandos según la dirección de la transmisión:

Dirección	TipoMensaje	Descripción / Propósito
Cliente → Servidor	CONECTAR	Solicita el ingreso a la partida con un nombre de jugador (DatosConectar).
	TIRAR_DADO	Envía el resultado físico de los dados leídos por el hardware (DatosTirarDado).
	COMPRAR_PROPIEDAD	Confirma la compra de la propiedad donde cayó el jugador.
	NO_COMPRAR	Rechaza la oferta de compra de la propiedad actual.
	PAGAR_DEUDA	Solicita el pago de deudas pendientes (rentas o impuestos).
	TERMINAR_TURNO	Finaliza formalmente el turno del jugador actual.
	CONSULTAR_TRANSACCIONES	Solicita la lista histórica de movimientos financieros.
	REGISTRAR_TARJETA	Vincula una tarjeta RFID a un jugador en la fase de registro (DatosTarjeta).
	TARJETA_RFID	Notifica la lectura de la tarjeta RFID durante el juego (DatosTarjeta).
	DESCONECTAR	Notifica la desconexión voluntaria del jugador.

Dirección	TipoMensaje	Descripción / Propósito
Servidor → Cliente	CONEXION_ACEPTADA	Confirma el ingreso y envía el estado inicial (DatosConexionAceptada).
	CONEXION_RECHAZADA	Rechaza el ingreso explicando el motivo (DatosConexionRechazada).
	ACTUALIZAR_ESTADO	Transmite el estado global actualizado de todos los jugadores (DatosActualizarEstado).
	TU_TURN0	Notifica al cliente especificado que es su turno de jugar (DatosTuTurno).
	RESULTADO_DADO	Notifica a todos los clientes el valor obtenido en los dados (DatosResultadoDado).
	OFERTA_COMPRA	Notifica que una casilla está disponible para compra (DatosOfertaCompra).
	HISTORIAL_TRANSACCIONES	Devuelve el listado de transacciones solicitadas (DatosHistorialTransacciones).
	EVENTO	Difunde descripciones de sucesos del juego para la bitácora (DatosEvento).
	ERROR	Informa un error de validación o regla de juego (DatosError).
	FIN_JUEGO	Declara el fin de la partida y anuncia al ganador (DatosFinJuego).

4. Estructuras de Datos / *Payloads* Destacados

- **JugadorEstado:** Objeto DTO que transporta la información de cada jugador por la red (ID, Nombre, Posición, Saldo, lista de IDs de propiedades, estado de bancarrota y registro de tarjeta RFID).
- **TransaccionInfo:** Estructura de transferencia de datos para la bitácora financiera que viaja entre el servidor y el cliente (ID, FechaHora, Turno, Tipo, Jugadores origen/destino, Monto y Descripción).